

Experiment - 6.1.1. Incremented Date

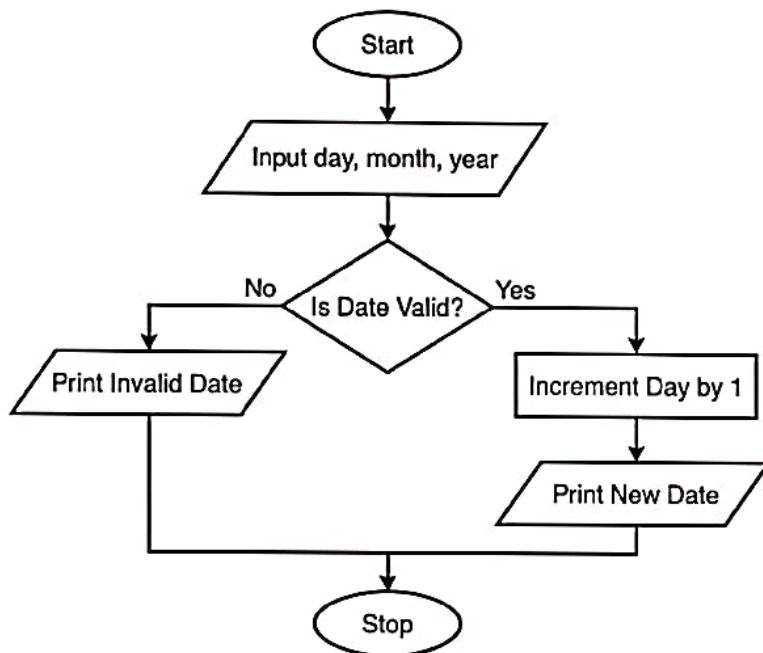
1. Aim

To design and implement a Python program that checks if a given date (day, month, year) is valid according to the Gregorian calendar rules. If the date is valid, the program computes and prints the next day's date (incremented date) in DD-MM-YYYY format. If the date is invalid, it prints "Invalid Date".

2. Pseudocode

1. **START**
2. **READ** three integers from the user representing the day, month, and year, and **STORE** them in variables d, m, and y.
3. **INITIALIZE** a boolean variable flag to True to keep track of date validity.
4. **CHECK** basic validity: **IF** $y \leq 0$ **OR** $m < 1$ **OR** $m > 12$, **SET** flag to False.
5. **IF** flag is True:
 - o **DETERMINE** maxdays based on the month:
 - **IF** m is 1, 3, 5, 7, 8, 10, or 12: maxdays = 31.
 - **ELSE IF** m is 4, 6, 9, or 11: maxdays = 30.
 - **ELSE IF** m is 2 (February):
 - **IF** the year y is a leap year ($(y \% 400 == 0)$ or $(y \% 4 == 0 \text{ and } y \% 100 \neq 0)$): maxdays = 29.
 - **ELSE**: maxdays = 28.
 - o **CHECK** day validity: **IF** $d < 1$ **OR** $d > \text{maxdays}$, **SET** flag to False.
6. **IF** flag is True (The date is completely valid):
 - o **INCREMENT** the day d by 1.
 - o **IF** $d > \text{maxdays}$ (End of the month reached):
 - **SET** d = 1.
 - **INCREMENT** the month m by 1.
 - **IF** $m > 12$ (End of the year reached):
 - **SET** m = 1.
 - **INCREMENT** the year y by 1.
 - o **PRINT** the incremented date formatted as DD-MM-YYYY (zero-padded).
7. **ELSE**:
 - o **PRINT** "Invalid Date".
8. **END**

3. Flowchart



4. Python Program

```
d = int(input())
m = int(input())
y = int(input())
```

```
flag = True
maxdays = 0
```

```
# Check valid year and month bounds
if y <= 0 or m < 1 or m > 12:
    flag = False
```

```
# Determine maximum days in the given month if year and month are valid
if flag:
    if m in (1, 3, 5, 7, 8, 10, 12):
        maxdays = 31
    elif m in (4, 6, 9, 11):
        maxdays = 30
    elif m == 2:
        # Leap year check
        if (y % 400 == 0) or (y % 4 == 0 and y % 100 != 0):
            maxdays = 29
        else:
```

```
    maxdays = 28

    # Check if the day is within the valid range for the month
    if d < 1 or d > maxdays:
        flag = False

# Increment date or print invalid message
if flag:
    d += 1
    # Check if we need to roll over to the next month
    if d > maxdays:
        d = 1
        m += 1
        # Check if we need to roll over to the next year
        if m > 12:
            m = 1
            y += 1

    # Print in zero-padded DD-MM-YYYY format
    print(f"{d:02d}-{m:02d}-{y:04d}")
else:
    print("Invalid Date")
```

Search course

- 1. Experiment - 1
- 2. Experiment - 2
- 3. Experiment - 3
- 4. Experiment - 4
- 5. Experiment - 5
- 6. Experiment - 6
- 6.1. Programs

6.1.1. Incremented Date

6.1.2. Factorial of a Number

- 7. Experiment - 7
- 8. Experiment - 8
- 9. Experiment - 9
- 10. Experiment - 10
- 11. Experiment - 11
- 12. Experiment - 12
- 13. Experiment - 13
- 14. Experiment - 14
- 15. Experiment - 15

6.1.1. Incremented Date

Write a Python program to check if a given date is valid. If the date is valid, print the next day's date (incremented date). If the date is invalid, print "Invalid Date".

Date Validation Rules:

- Valid Month:** 1 to 12
- Valid Day:** Depends on the month and year
 - January (1), March (3), May (5), July (7), August (8), October (10), December (12): 1 to 31 days
 - April (4), June (6), September (9), November (11): 1 to 30 days
 - February (2): 1 to 29 days in a leap year and 1 to 28 days in a non-leap year
- Valid Year:** Any positive integer greater than zero.

Date Increment Rules:

- If the date is valid, increment it by one day:
- If it's the last day of the month, move to the 1st day of the next month
 - If it's December 31st, move to January 1st of the next year

Input Format:

- First line contains an integer representing the day
- Second line contains an integer representing the month
- Third line contains an integer representing the year

Output Format:

- If the date is valid, print the incremented date in the format:

<DD>-<MM>-<YYYY>

- If the date is invalid, print: "Invalid Date".

Sample Test Cases

+

nextDate.py

```
1 d = int(input())
2 m = int(input())
3 y = int(input())
4
5 flag = True
6
7 if y < 1 or m < 1 or m > 12:
8     flag = False
9 else:
10     if m in (1, 3, 5, 7, 8, 10, 12):
11         maxdays = 31
12     elif m in (4, 6, 9, 11):
13         maxdays = 30
14     elif m == 2:
15
16         if (y % 400 == 0) or (y % 4 == 0 and y % 100 != 0):
17             maxdays = 29
18         else:
19             maxdays = 28
20     else:
```

Average time
0.007 s
7.40 ms

Maximum time
0.015 s
15.00 ms

5 out of 5 shown test case(s) passed
5 out of 5 hidden test case(s) passed

Test case 1 15 ms

Expected output

15

3

2024

16-03-2024

Actual output

15

3

2024

16-03-2024

Debug

Terminal

Test cases

Prev

Reset

Submit

Next